

第23回 実践：Web ゲームプログラミング2（キー入力）

- 前回：ゲームを自動で動かし続けた
- 今回：キーボードで自機を動かす
- 次回：自機から弾を撃つ

実習時のフォルダ構成

Web_名前 の中に 23 フォルダを作る。第 22 回の sample フォルダと exercise フォルダを、23 の中にコピーして、続きから作る。

```
Web_名前
├── 23
│   ├── sample          ← 第 22 回の sample をコピーする
│   │   ├── index.html
│   │   ├── style.css
│   │   └── main.js
│   └── exercise        ← 第 22 回の exercise をコピーする
│       ├── index.html
│       ├── style.css
│       └── main.js
```

- sample：講義で使うサンプル（講義内容のコードは、sample/main.js に書き足す）
- exercise：自分のシューティングゲーム（演習 1～2、発展）
- index.html と style.css は、第 22 回のままで変更しない。

講義内容

sample/main.js に書き足す。書いたら sample/index.html をブラウザで開いて確認する。キー入力を確認するときは、ブラウザの画面を一度クリックしてから、キーを押す。

1. 前回の復習

- ゲームループ：gameLoop の中で「update（更新）→ draw（描画）→ requestAnimationFrame(gameLoop)」を繰り返す。約 60 回／秒。
- const の player でも、player.x などの中の値は書き換えられる。

今回は、キーボードの ← → キーで、自機を左右に動かす。

2. 今回の main.js の並び

今回は「キー入力」の部分を追加し、updatePlayer を書き換える。

```
// ===== 準備 =====
// ===== ゲームの状態 =====
// ===== 自機 =====
// ===== キー入力 =====    ← 今回追加する
// ===== 更新 =====        ← updatePlayer を書き換える
// ===== 描画 =====
// ===== ゲームループ =====
```

3. 第 20 回のクリックと、今回のキー入力

第 20 回では、ボタンがクリックされたら関数を呼び出した。

```
scoreButton.addEventListener("click", addTenPoints);
```

今回は、キーが押されたら処理をする。書き方は次のようになる。

```
document.addEventListener("keydown", function (event) {
  キーが押されたときに実行する処理
});
```

「イベント（出来事）が起きたら、関数を呼び出す」という考え方は、第 20 回と同じ。違うのは次の 3 点。

	第 20 回（クリック）	今回（キー入力）
どこで待つ か	ボタン（ <code>scoreButton</code> ）	ページ全体（ <code>document</code> ）
何を待つ か	<code>"click"</code>	<code>"keydown"</code> （キーが押された）、 <code>"keyup"</code> （キーが離れた）
呼び出す関 数	名前のある関数 （ <code>addTenPoints</code> ）	その場で書く、 名前のない関数 <code>function (event) { }</code>

- `function (event) { }` は、名前のない関数。そこでしか使わない関数なので、名前を付けずに、その場で作って `addEventListener` に渡している。
- イベントが起きるたびに、ブラウザがこの関数を呼び出す。そのとき、引数 `event` に、イベントの情報が入る。
- 今回は、`event` の中の `event.key`（押されたキーの名前）だけを使う。

キー	<code>event.key</code> の値
← → ↑ ↓	<code>"ArrowLeft"</code> <code>"ArrowRight"</code> <code>"ArrowUp"</code> <code>"ArrowDown"</code>
スペース	<code>" "</code> （空白 1 文字）
A	<code>"a"</code>

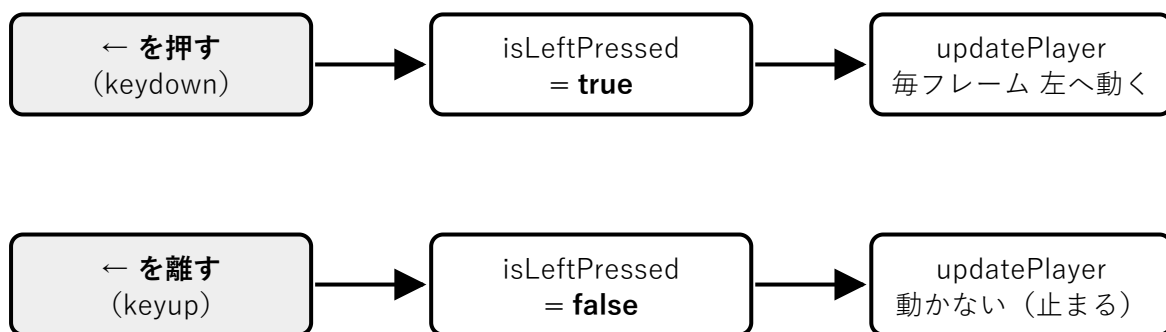
4. キーの状態を変数に記録する

「自機」の部分の下に、「キー入力」の部分を追加する。

```
// ===== キー入力 =====
let isLeftPressed = false;
let isRightPressed = false;

document.addEventListener("keydown", function (event) {
  console.log(event.key); // 押したキーの名前を確認する
  if (event.key === "ArrowLeft") {
    isLeftPressed = true;
  }
  if (event.key === "ArrowRight") {
    isRightPressed = true;
  }
});

document.addEventListener("keyup", function (event) {
  if (event.key === "ArrowLeft") {
    isLeftPressed = false;
  }
  if (event.key === "ArrowRight") {
    isRightPressed = false;
  }
});
```



イベントでは「押されているか」を記録するだけ。動かすのはゲームループの `updatePlayer`。

- キーを押したときは、自機を直接動かさず、「押されているかどうか」だけを変数に記録する。こうすると、押している間、毎フレームなめらかに動き続ける。
- `console.log(event.key);` で、押したキーの名前がコンソールに表示される。いろいろなキーを押して確認する（この行は、次回、書き換えるときに消す）。

5. updatePlayer を書き換える

「更新」の部分の `updatePlayer` を、次のように書き換える（第 22 回の「右端から出たら左端に戻す」処理は消す）。

```
function updatePlayer() {
  if (isLeftPressed) {
    player.x -= player.speed;
  }
  if (isRightPressed) {
    player.x += player.speed;
  }
  // 画面の端で止める
  if (player.x < 0) {
    player.x = 0;
  }
  if (player.x + player.w > canvas.width) {
    player.x = canvas.width - player.w;
  }
}
```

- ← が押されていれば左へ、→ が押されていれば右へ、`speed` だけ動かす。
- 第 18 回で学んだ「画面の端で止める」処理。右端は、自機の右端 (`player.x + player.w`) で比べる。

6. うまく動かないとき

症状	確認すること
キーを押しても動かない	画面を一度クリックしたか。"ArrowLeft" の綴り (大文字・小文字)。コンソールに <code>event.key</code> が出ているか
離しても止まらない	<code>keyup</code> のほうで <code>false</code> に戻しているか。キーを押したままブラウザの外をクリックした場合は、画面をクリックしてから、同じキーをもう一度押して離す
画面の外に出る	端で止める <code>if</code> があるか。右端は <code>player.w</code> を足して比べているか

演習

`exercise/main.js` (自分のシューティングゲーム) に書く。

演習 1 キーで自機を左右に動かす

- 講義と同じように「キー入力」の部分を追加し、`updatePlayer` を書き換えて、← → キーで自機が左右に動くようにする。
- 左右の端で、自機が止まることを確認する。

演習 2 別のキーでも動かす

- `console.log(event.key)` で、A キーと D キーの名前を確認する。
- A キーでも左へ、D キーでも右へ動くようにする。(ヒント: `keydown` と `keyup` の両方の条件を、`||` を使って書き換える。)

発展（任意）

-   キーで、自機が上下にも動くようにする。上下の端でも止まるようにする。

この回の終わりの `sample/main.js` 全体

```
// ===== 準備 =====
const canvas = document.getElementById("game");
const ctx = canvas.getContext("2d");

// ===== ゲームの状態 =====
let frameCount = 0;           // ゲーム開始からのフレーム数

// ===== 自機 =====
const player = { x: 184, y: 440, w: 32, h: 32, speed: 5 };

// ===== キー入力 =====
let isLeftPressed = false;
let isRightPressed = false;

document.addEventListener("keydown", function (event) {
  console.log(event.key);           // 押したキーの名前を確認する
  if (event.key === "ArrowLeft") {
    isLeftPressed = true;
  }
  if (event.key === "ArrowRight") {
    isRightPressed = true;
  }
});

document.addEventListener("keyup", function (event) {
  if (event.key === "ArrowLeft") {
    isLeftPressed = false;
  }
  if (event.key === "ArrowRight") {
    isRightPressed = false;
  }
});

// ===== 更新 =====
function updatePlayer() {
  if (isLeftPressed) {
    player.x -= player.speed;
  }
  if (isRightPressed) {
    player.x += player.speed;
  }
  // 画面の端で止める
  if (player.x < 0) {
    player.x = 0;
  }
  if (player.x + player.w > canvas.width) {
    player.x = canvas.width - player.w;
  }
}

function update() {
  frameCount += 1;
  updatePlayer();
}
```

```
// ===== 描画 =====
function drawBackground() {
  ctx.fillStyle = "#000022";
  ctx.fillRect(0, 0, canvas.width, canvas.height);
}

function drawPlayer() {
  ctx.fillStyle = "#00ccff";
  ctx.fillRect(player.x, player.y, player.w, player.h);
}

function draw() {
  drawBackground();
  drawPlayer();
}

// ===== ゲームループ =====
function gameLoop() {
  update();
  draw();
  requestAnimationFrame(gameLoop);
}

gameLoop();
```